

1. Conceptos Teóricos Fundamentales

El "Tridente" CIA (Seguridad de la Información)

- **Confidencialidad (*Confidentiality*):** Propiedad que garantiza que la información llegue solamente a las personas autorizadas e impide la divulgación a otras personas o sistemas no autorizados.
- **Integridad (*Integrity*):** Propiedad que mantiene con exactitud y fidelidad los datos, asegurando que no sean modificados de forma no autorizada.
- **Disponibilidad (*Availability*):** Los servicios deben estar siempre activos y funcionando correctamente para que los usuarios puedan usarlos siempre que lo requieran.

Tipos de Copias de Seguridad

- **Completa:** Se copian absolutamente todos los datos del sistema o base de datos. Es la base de cualquier estrategia de backup.
- **Diferencial:** Se copian únicamente las diferencias/cambios respecto a la **última copia de seguridad completa**.
- **Incremental:** Se copian las diferencias/cambios respecto a la **última copia de seguridad realizada** (ya sea esta completa o incremental).

2. Copias de Seguridad Lógicas (*mysqldump*)

Para crear la copia lógica se utiliza el comando *mysqldump*. El archivo resultante es un script con sentencias SQL (*.sql*).

Sintaxis para la Creación de Backups

```
# 1. Copiar una base de datos específica (Sin comandos CREATE DATABASE/USE)
mysqldump -h IP -u ELUSUARIO -p banco > backup_banco.sql

# 2. Copiar solo tablas específicas de una base de datos
mysqldump -h IP -u ELUSUARIO -p banco cliente cuenta > backup_tablas.sql

# 3. Copiar varias bases de datos específicas (Incluye comandos CREATE DATABASE y USE)
mysqldump -h IP -u ELUSUARIO -p --databases bd_juego banco > copia_multidb.sql

# 4. Copia de seguridad total de todo el servidor MariaDB/MySQL
mysqldump -h IP -u ELUSUARIO -p --all-databases > copia_total.sql
```

Restauración de Copias Lógicas

Como las copias lógicas son scripts de SQL puro, para restaurarlas basta con ejecutarlas en el servidor de destino.

Caso A: Desde la consola interactiva de MariaDB

```
mysql> source micopia.sql
```

Caso B: Desde la terminal de comandos (CMD o Linux)

```
mysql -u ELUSUARIO -h IP -p < micopia.sql
```

Caso C: Si la copia NO incluye **CREATE DATABASE** y **USE**

Si se exportó una sola base de datos sin la opción **--databases**, el archivo carece de las instrucciones de creación. Se puede resolver inyectando el comando inicial:

```
mysql -u ELUSUARIO -h IP -p --init-command="CREATE DATABASE IF NOT EXISTS  
nombredelabdd; USE nombredelabdd;" < micopia.sql
```

Caso D: Restaurar una única base de datos cuando hay varias en el archivo **.sql**

Utilizando el parámetro **--one-database**.

- *Inconveniente*: Si alguna de las otras bases de datos no existe en el servidor, el comando la creará vacía (sin tablas).

```
mysql -u ELUSUARIO -h IP -p --one-database banco < micopia.sql
```

3. Copias de Seguridad Físicas (**mariadb-backup**)

Copia directamente los archivos binarios y directorios del motor de datos. Es muchísimo más rápido y eficiente en bases de datos gigantescas.

Paso 1: Realizar el Backup Completo Inicial

```
sudo mariadb-backup --backup \  
--target-dir=/home/osboxes/copiacompleta \  
--user="root" -p
```

Paso 2: Realizar un Backup Incremental

Guarda únicamente lo que cambió desde la base especificada en `--incremental-basedir`.

```
sudo mariadb-backup --backup \  
  --target-dir=/home/osboxes/inc_martes \  
  --incremental-basedir=/home/osboxes/copiacompleta \  
  --user="root" -p
```

Paso 3: PREPARAR (Fusión de archivos)

Antes de restaurar una copia física, se deben consolidar y aplicar los archivos incrementales sobre el backup completo.

```
# 1. Preparar la copia completa inicial (Se recomienda usar --apply-log-only si se  
van a fusionar más incrementales)  
sudo mariadb-backup --prepare --target-dir=/home/osboxes/copiacompleta  
  
# 2. Fusionar el incremental dentro de la copia completa  
sudo mariadb-backup --prepare \  
  --target-dir=/home/osboxes/copiacompleta \  
  --incremental-dir=/home/osboxes/inc_martes
```

4. Administración, Bloqueos y Recuperación Avanzada

Mantenimiento de Bases de Datos

```
DROP DATABASE IF EXISTS base1;  
DROP DATABASE IF EXISTS base2;
```

Bloqueos de Seguridad para Backups Manuales

Evita que se escriban datos mientras se realiza una copia manual y permite conocer el estado exacto del log binario.

```
-- 1. Bloquear tablas en modo lectura  
FLUSH TABLES WITH READ LOCK;  
  
-- 2. Ver posición actual del log binario (indispensable para Point-in-Time  
Recovery)  
SHOW MASTER STATUS;  
  
-- 3. Liberar el bloqueo una vez terminado el proceso
```

```
UNLOCK TABLES;
```

Recuperación por Log Binario (Point-in-Time Recovery)

Permite recuperar transacciones exactas que ocurrieron entre el último backup y el momento justo del fallo del sistema.

```
# Inyectar el log binario filtrado por posiciones directamente al cliente mysql
sudo mysqlbinlog --start-position=123 --stop-position=456
/var/lib/mysql/logbinario.000002 | mysql -u usuario -p
```

Dentro del flujo de comandos, desactivar temporalmente el log binario (`SET sql_log_bin=OFF;`) previene bucles infinitos de reescritura al restaurar.

5. Programación con Bases de Datos (MariaDB / MySQL)

Almacenamiento de Información en Memoria (Variables de Usuario)

```
-- Definición y asignación de variables (anteponiendo el carácter @)
SET @nombre = "NombreUsuario";
SET @digito = 1234567890;
SET @fecha = '2026-04-30';

-- Salida de contenido por consola
SELECT @nombre;

-- Modificación de valores aritméticos y asignación de nulos
SET @digito = @digito + 3;
SET @fecha = NULL;
```

Estructuras Condicionales (IF - ELSEIF - ELSE)

```
SET @num1 = 1;
SET @num2 = 7;
SET @test = NULL;

IF @num1 = @num2 THEN
    SET @test = "Salida1";
ELSEIF @num1 > @num2 THEN
    SET @test = "Salida2";
ELSEIF @num1 < @num2 THEN
    SET @test = "Salida3";
ELSE
```

```
SET @test = "Salida por defecto";  
END IF;
```

Uso de Delimitadores y Estructuras Repetitivas (WHILE)

Las sentencias que no van tabuladas/anidadas dentro de un bloque estructurado se cierran con el nuevo delimitador personalizado (//), mientras que las internas mantienen el punto y coma ;.

```
-- Ejemplo 1: Verificación de año bisiesto mediante script aislado  
DELIMITER //  
SET @a = 2026//  
IF (@a % 4 = 0 AND @a % 100 != 0) OR (@a % 400 = 0) THEN  
    SELECT 'Bisiesto';  
ELSE  
    SELECT 'No Bisiesto';  
END IF//  
DELIMITER ;  
  
-- Ejemplo 2: Bucle iterativo descendente  
DELIMITER //  
SET @x = 10//  
WHILE @x > 0 DO  
    SET @x = @x - 1;  
    SELECT @x;  
END WHILE//  
DELIMITER ;
```

Creación y Uso de Funciones Stored (CREATE FUNCTION)

1. Función Básica: Suma de dos enteros

```
DELIMITER //  
CREATE FUNCTION suma(x INT, y INT) RETURNS INT  
BEGIN  
    RETURN x + y;  
END//  
DELIMITER ;  
  
-- Llamadas y uso de la función  
SELECT suma(15, 2);  
SET @resultado = suma(15, -5);  
SELECT suma(suma(10, 5), 15);
```

2. Función Determinista: Comprobación de Año Bisiesto

(Se añade bloque **BEGIN ... END** obligatorio al declarar variables internas)

```
DELIMITER //
CREATE OR REPLACE FUNCTION esbisiesto(anyo INT) RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE resultado BOOLEAN;
    IF (anyo % 4 = 0 AND anyo % 100 != 0) OR (anyo % 400 = 0) THEN
        SET resultado = TRUE;
    ELSE
        SET resultado = FALSE;
    END IF;
    RETURN resultado;
END//
DELIMITER ;
```

3. Función Determinista: Divisibilidad de dos números

```
DELIMITER //
CREATE OR REPLACE FUNCTION esDivisible(digit INT, digit2 INT) RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE resultado2 BOOLEAN;
    IF (digit % digit2 = 0) THEN
        SET resultado2 = TRUE;
    ELSE
        SET resultado2 = FALSE;
    END IF;
    RETURN resultado2;
END//
DELIMITER ;

-- Pruebas de validación de retornos
SELECT esDivisible(4, 2); -- Retorna TRUE (1)
SELECT esDivisible(2, 2); -- Retorna TRUE (1)
SELECT esDivisible(15, -3); -- Retorna TRUE (1)
SELECT esDivisible(8, 3); -- Retorna FALSE (0)
```

4. Función de Lectura de Datos (READS SQL DATA)

Función que interactúa con tablas de la base de datos para comprobar si existen huecos secuenciales en las llaves primarias.

```
DELIMITER //
CREATE OR REPLACE FUNCTION hayHuecos() RETURNS BOOLEAN
```

```
READS SQL DATA
BEGIN
    DECLARE v_contador INT DEFAULT 0;
    DECLARE v_maxid INT DEFAULT 0;

    SELECT COUNT(*), MAX(id_plataforma)
    INTO v_contador, v_maxid
    FROM bd_juego.plataforma;

    RETURN v_contador < v_maxid;
END//
DELIMITER ;
```